Sorbonne Université

Faculté des Sciences et Ingénierie

Projet IREC

Combinaison d'optimisation de trajectoires et d'optimisation de politique en apprentissage par renforcement

Sujet 3

Réalisé par :

Théo-Paul LADET
Riadh OUAZIB
Thomas Lu

Encadrant :

Pr. Olivier Sigaud

Année Universitaire : 2025–2026

Département Informatique

Abstract

This synthesis document is part of the "Topic 3" project focusing on the combination of trajectory optimization (MPC) and policy optimization (RL). The central question is: *How can we combine and ensure synergy between the complementary processes of trajectory optimization and policy optimization within a single architecture, by implementing effective alignment mechanisms?* We perform an analysis of four recent architectures: LOOP (Sikchi et al.), BMPC (Wang et al.), TD-M(PC)^2 (Lin et al.), and AC-MPC (Romero et al.). We specifically study how each method attempts to align the behaviour of the neural network with that of the planner, highlighting the trade-offs between explicit constraints, imitation, and end-to-end learning. We pay particular attention to the AC-MPC architecture, which illustrates a new way of looking at the usual directionality in Policy–MPC alignment. This report details the mathematical formulations, compares empirical performance, and discusses the theoretical implications of these alignment mechanisms.

Contents

1 Introduction: Combining Policy and Planning, The Alignment Problem **1**
- 1.1 Project Objective . . . 1
- 1.2 Algorithms Studied . . . 1

2 Unified Theoretical Framework **2**
- 2.1 The Hybrid Optimization Problem . . . 2

3 Analysis of Alignment Mechanisms **2**
- 3.1 LOOP: Planner-Constrained Alignment . . . 2
  - 3.1.1 Alignment Mechanism: Actor Regularized Control (ARC) . . . 3
  - 3.1.2 Implementation and Analysis . . . 3
- 3.2 TD-M(PC)^2 : Alignment via Learning Constraint . . . 4
- 3.3 BMPC: Alignment through Imitation and Bootstrapping . . . 5
  - 3.3.1 Alignment Mechanism: Iterative Expert Imitation . . . 5
  - 3.3.2 Model-Based Value Learning and Lazy Reanalyze . . . 5
- 3.4 AC-MPC: Alignment via Differentiation . . . 6
  - 3.4.1 Architecture and operation . . . 6
  - 3.4.2 Key contribution: Model-Predictive Value Expansion (MPVE) . . . 6
  - 3.4.3 Algorithme AC-MPC . . . 6

4 Comparison and Discussion **9**

5 Conclusion **9**

1 Introduction: Combining Policy and Planning, The Alignment Problem

Reinforcement Learning (RL) aims to optimize the sum of rewards received by an agent interacting with an environment. This optimization can follow two main, often considered orthogonal, routes:

- **Policy Optimization:** Learning a parametric policy $\pi_\theta(a \mid s)$ (often a neural network) that directly maps a state to an action. This "model-free" or "model-based policy optimization" approach seeks to maximize the expected future rewards via gradient descent or evolutionary methods.
- **Trajectory Optimization:** Using a model of the environment $f(s, a)$ to plan an optimal sequence of actions over a finite horizon. This is the domain of Model Predictive Control (MPC).

From a robotics control perspective, these two paradigms are deeply complementary. MPC provides online replanning capabilities, exploits an explicit dynamic model, and handles input constraints, making it a very robust tool for stabilization and safety. This comes at the cost of a short planning horizon and a cost function that is often hand-crafted, conservative, and task-specific.

Conversely, policy optimization via RL can integrate very general objectives, potentially non-differentiable and sparse (lap time, collisions, energy, etc.), and reason over the long term via the value function. However, it suffers from out-of-distribution robustness issues, massive data requirements, and a lack of stability guarantees.

In recent years, hybrid algorithms have attempted to combine these two approaches to leverage both the generalization power of RL and the local accuracy of MPC. The idea is to use RL to learn a terminal value function $V(s)$ that guides MPC beyond its planning horizon, or to use MPC to generate high-quality data to train the RL policy.

However, this fusion raises a fundamental challenge: synergy and alignment. If the planner (MPC) and the learned policy (RL) are not "aligned", they risk working against each other, leading to instabilities or sub-optimal performance. This phenomenon manifests through several pathologies:

- **Distributional Shift**: MPC may explore states that the neural network has never seen, making value estimates ($V(s)$ or $Q(s, a)$) invalid.
- **Value Overestimation**: The neural network may provide incorrect, overly optimistic value estimates for certain actions, misleading the planner, which will exploit these errors ("delusional bias").
- **Actor Divergence**: The implicit policy of MPC and the explicit policy of the neural network can diverge, making it ineffective to learn one from the other.

1.1 Project Objective

This document focuses on the study of the mechanisms used to guarantee behavioral alignment. We describe the algorithms and dissect how they enforce, encourage, or learn this alignment.

1.2 Algorithms Studied

We analyze four major and recent contributions (2022–2025) that represent the state of the art on this topic:

1. **BMPC (Bootstrapped MPC)** [1]: An approach based on iterative imitation.
2. **LOOP (Learning Off-Policy with Online Planning)** [2]: An approach based on constraining the planner.
3. **TD-M(PC)^2** [3]: An approach based on constraining value learning.
4. **AC-MPC (Romero et al.)** [4]: An end-to-end differentiation-based approach (analyzed as a complementary method for its unique perspective).

2 Unified Theoretical Framework

To compare these methods, we define a common formal framework. We consider a Markov Decision Process (MDP) defined by $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$.

2.1 The Hybrid Optimization Problem

The objective is to find a sequence of actions $u_{0:H-1}$ that maximizes:

$$J = \sum_{t=0}^{H-1} \gamma^t r(s_t, u_t) + \gamma^H V_\phi(s_H), \tag{1}$$

where:

- H is the MPC planning horizon.
- $r(s, u)$ is the terminal value function, parameterized by ϕ, learned by RL.

At each time step, the MPC planner solves:

$$\pi_{\text{MPC}}(s) = \arg\max_{u_0} \max_{u_{1:H-1}} \mathbb{E}\left[\sum_{t=0}^{H-1} \gamma^t r(s_t, u_t) + \gamma^H V_\phi(s_H)\right]. \tag{2}$$

The challenge is to ensure that V_ϕ is an accurate estimate of the true value under the MPC policy, and that MPC does not move too far into regions where V_ϕ is invalid.

3 Analysis of Alignment Mechanisms

We now analyze how each algorithm tackles this alignment problem.

3.1 LOOP: Planner-Constrained Alignment

The LOOP algorithm (Sikchi et al., 2022) introduces a specific instantiation of the so-called *H-step lookahead policy.*
Rather than choosing between a reactive policy (RL) and long-horizon planning (MPC), LOOP combines both:

- The algorithm learns a set of probabilistic dynamics models to capture epistemic uncertainty. This allows simulating the environment over a short horizon H.
- To estimate rewards beyond horizon H, LOOP uses a value function learned by an off-policy algorithm (here, Soft Actor-Critic).

At each time step, the planner solves the following optimization problem to find the optimal sequence $u_{0:H-1}$:

$$J_{\text{LOOP}} = \mathbb{E}\left[\sum_{t=0}^{H-1} \gamma^t r(s_t, u_t) + \gamma^H V_\phi(s_H)\right] \tag{3}$$

In practice, LOOP operates iteratively: the planner (MPC guided by V_ϕ) interacts with the environment to collect high-quality data. This data is stored in a replay buffer and used to train the actor π_θ and the critic V_ϕ via SAC, as well as to update the dynamics model.

The authors formally show that the performance gap of an H-step lookahead policy is bounded by two terms:

- The model error, which accumulates over horizon H.
- The value function error, attenuated by a factor γ^{H-1}.

This justifies the LOOP architecture: when the value function is imperfect (which is always the case in deep RL due to bias and variance), using a planning horizon H reduces the dependence on that erroneous value. Conversely, using a terminal value makes it possible to keep H short enough to avoid model divergence.

3.1.1 Alignment Mechanism: Actor Regularized Control (ARC)

However, this separation of tasks creates a structural flaw identified by the authors as *Actor Divergence*.

By aggressively maximizing reward, the MPC planner tends to generate trajectories that drift away from the data distribution seen by the actor during training. Yet the value function V_ϕ is only valid on the distribution induced by π_θ. If MPC drives the system into regions unknown to the actor (out-of-distribution), the estimate V_ϕ becomes incorrect (often overestimated), which traps the planner in sub-optimal actions.

To address this, LOOP proposes to modify the MPC objective to include a proximity constraint to the learned policy π_θ. The optimization problem becomes:

$$\max_{\tau} \ \mathbb{E}[J(\tau)] \quad \text{s.t.} \quad D_{\text{KL}}(p(\tau) \,\|\, \pi_\theta(\tau)) \leq \varepsilon \tag{3}$$

where $p(\tau)$ is the trajectory distribution of the planner and $\pi_\theta(\tau)$ that induced by the neural actor.

The optimal solution has the form of a tilted Boltzmann distribution:

$$p^*(\tau) \propto \pi_\theta(\tau) \, \exp\left(\frac{1}{\eta} J(\tau)\right) \tag{4}$$

where η is a Lagrange multiplier (a temperature parameter).

3.1.2 Implementation and Analysis

The **directionality** of alignment is thus **Policy $\rightarrow$ MPC**.

In practice, LOOP implements this via a CEM (Cross-Entropy Method) or MPPI (Model Predictive Path Integral)-type method, where the sampling distribution is biased towards π_θ.

- **Strength**: This ensures that MPC explores around a known "anchor" (the policy), avoiding out-of-distribution regions where V_ϕ would be inaccurate.
- **Weakness**: In return, this potentially restricts MPC's ability to find solutions that are radically better than the current policy. MPC becomes a "local improver" of the policy.

3.2 TD-M(PC)^2 : Alignment via Learning Constraint

Model based reinforcement learning has seen considerable progress thanks to hybrid algorithms that combine both planning and learning. Algorithms like TD MPC stand out by combining a planner (MPC), that predicts the most optimal action over a small horizon, with a learned value (TD) function to see beyond the horizon.

TD MPC2, which is an upgrade to TD MPC, demonstrates robust performances on various continuous control benchmarks. Its ability to use one single set of hyperparameter for diverse tasks makes it a reference solution for general robotic controls.

However, the upgrade does not solve the initial problem that TD MPC has introduced. It performs poorly on high dimensional locomotion tasks like the control of a humanoid robot with 61 degrees of liberty (61 – DoF), which is due to the policy mismatch that was introduced in TD MPC.

In TD MPC, the data are collected by the planner which are then used to train the policy network. The problem is that, in most cases, the planner is too effective, which results in a value overestimation by the policy network as it rarely sees the consequences of its bad choices. This mismatch prevents the policy network from correcting its wrong estimations, which creates a vicious cycle on complex task learning.

To mitigate this policy mismatch, we have two choices: introducing constraints on the planner or on the policy network to align both policies. The first method was used in LOOP, however by introducing constraints on the planner, it harms exploration and data efficiency which leads to poor performances. That is why TD (MPC)^2, which uses constraints on the policy network, has been introduced.

TD (MPC)^2 is a direct upgrade to TD MPC2 which tries to mitigate the policy mismatch problem while keeping its original performance.

To avoid out-of-distribution query, a constrained policy improvement can be described as solving a constrained optimization problem:

$$\pi_{k+1} := \arg\max_{\pi} \mathbb{E}_{a\sim\pi(\cdot|s)}\left[Q_k(a,s)\right] \quad (1)$$
$$\text{s.t.} \quad D_{KL}\left(\pi_{k+1} \| \mu_k\right) \leq \epsilon$$

Where μ_k denotes the behavior policy from the buffer at the k-th iteration. It can be represented as a weighted sum of H-step lookahead policies of past iteration:

$$\mu_K(\cdot|s) = \sum_{k=0}^{K} \omega_k \pi_{H,k}(\cdot|s),$$

where ω_k is the weight for the multivariant Gaussian policy $\pi_{H,k}$ and D_{KL} is the Kullback-Leibler divergence.

Instead of solving (1), we can instead minimize the policy loss function:

$$\mathcal{L}_\pi = -\mathbb{E}_{a\sim\pi}\left[Q(s,a) - \alpha \log \pi(a|s) + \beta \log \mu(a|s)\right]$$

(A) (B) (C)

(A) + (B) + (C) is a real random variable thanks to the linearity and all the functions being measurable on $(\mathbb{R}^n, \mathcal{B}(\mathbb{R}^n))$, therefore, the loss function is well defined.

If $X \leq Y$ almost surely then we have: $\mathbb{E}[X] \leq \mathbb{E}[Y]$

Therefore, the loss function can be interpreted as follows:

Minimizing the function results in maximizing what is inside the expectation.

We want to maximize (A) so the policy is trained to give the best actions.

As for (B), α is the entropy coefficient which is a positive number so we want to minimize $\log \pi(a|s)$ which is the π -log-likelihood of a, which means that it is at worst equal to 0. Therefore, we want to minimize $\pi(a|s)$ so π remains a probability.

As for (C), as β, which is the policy network coefficient, is a positive number, we want to maximize $\log \mu(a|s)$ but again as it is the log-likelihood of a, we need $\log \mu(a|s)$ to be as close to 0 as possible and that is done by having the policy as alike as the planner as possible.

Empirically, TD-M(PC)^2 demonstrates superior data efficiency and asymptotic performance compared to baselines, particularly in high-dimensional settings like the 61-DoF Humanoid tasks, effectively mitigating value overestimation even if it only partially addresses the root causes of model bias.

However, relying on a strict policy constraint creates a new bottleneck where the learned network (the student) becomes artificially bounded by the planner's capabilities (the teacher), preventing the policy from discovering optimal strategies that might lie outside the planner's distribution.

3.3 BMPC: Alignment through Imitation and Bootstrapping

Wang et al. (2025) [1] introduced Bootstrapped Model Predictive Control (BMPC), which adopts a fundamentally different approach to the alignment problem. The authors observe that in conventional model-free approaches combined with MPC (such as TD-MPC2), policy learning often significantly lags behind the planner's performance. This performance gap results in a weak terminal value function, which deteriorates the MPC planner's effectiveness and creates an inefficient cycle.

Rather than artificially constraining the planner (as in LOOP) or the value update (as in TD-M(PC)^2), BMPC embraces the planner's superior performance. It treats the MPC planning process as an expert to be imitated, rather than merely as a control mechanism.

3.3.1 Alignment Mechanism: Iterative Expert Imitation

The core mechanism of BMPC is a bootstrapped loop where the network policy π_θ learns to clone the behavior of the MPC planner. This is formalized by minimizing the Kullback-Leibler divergence between the network policy and the MPC distribution:

$$\mathcal{L}_p(\theta) = \mathbb{E}_{s\sim\mathcal{B}}\left[KL(\pi_{\text{MPC}}(\cdot|s)||\pi_\theta(\cdot|s))\right] \tag{4}$$

This implies a directionality of **Planner** $\rightarrow$ **Policy**. By imitating the MPC, the network policy π_θ improves rapidly. Crucially, this improved policy is then fed back into the MPC as a prior (or guide) for the sampling distribution (MPPI). This creates a virtuous cycle:

1. The policy guides the MPC, making planning more efficient and focused.
2. The MPC refines this guidance to produce superior actions.
3. The policy imitates these superior actions to improve further.

3.3.2 Model-Based Value Learning and Lazy Reanalyze

To support this process, BMPC abandons off-policy Q-learning (which suffers from distribution shift) in favor of on-policy, model-based Temporal Difference (TD) learning. The value target is computed using the learned world model and the current policy π_θ:

$$\hat{V}(s_t) = r_t + \gamma V_\phi(s_{t+1}) \quad \text{where } s_{t+1} \sim \text{Model}(s_t, \pi_\theta(s_t))$$

A major practical innovation in BMPC is the **Lazy Reanalyze** mechanism. Since re-planning with MPC for every training sample is computationally prohibitive, BMPC maintains

an "expert dataset" in the replay buffer. It periodically (but infrequently, e.g., every 10 updates) re-runs the planner on stored states to update the expert targets. This ensures the imitation targets remain fresh and aligned with the current capabilities of the agent, without the massive computational cost of full expert iteration.

The results presented in [1] show that this approach drastically reduces the performance gap between the neural policy and the planner, particularly in high-dimensional tasks like the "Dog Run" or "Humanoid Run" benchmarks, where standard methods often fail to align the two components.

3.4 AC-MPC: Alignment via Differentiation

In the architectures discussed above, the mechanisms used to address the alignment problem all exhibit a certain directionality in how control flows between the actor and the planner.

The LOOP algorithm uses the actor to control the planner by restricting it, via the policy, to trust regions. Directionality: Policy → Planner.

TD-(MPC)^2 and BMPC, on the other hand, use the planner to control the policy, through distribution regularization and imitation, respectively. Directionality: Planner → Policy.

The paper *Actor-Critic Model Predictive Control* (AC-MPC) [4] introduces a methodological shift by replacing the last layer of a classical actor network with an optimization module.

3.4.1 Architecture and operation

In a standard Actor–Critic scheme (such as PPO), the actor is a multilayer perceptron (MLP) that takes the state as input and outputs an action. In AC-MPC:

1. **The actor (cost map)**: the neural network takes the state s_t as input but does not output an action. It instead generates a local cost function, parameterized by a matrix $Q(s_t)$ and a vector $p(s_t)$. This is a way of encoding the immediate objective ("fly fast", "avoid this obstacle") in a mathematical form that a solver can exploit.

2. **The differentiable MPC module**: this module receives the generated cost function and uses the robot dynamics model $f(x, u)$ to solve the optimization problem

$$u^* = \arg\min_u \sum_{k=0}^{T} \left(x_k^\top Q x_k + p^\top x_k \right).$$

   The optimal action u^* is then applied to the robot.

3. **End-to-end learning**: the key feature is the use of automatic differentiation through the optimizer (via KKT conditions or the implicit function theorem). This means that if the robot fails, the algorithm can compute "how to modify the matrix Q so that the action u^* would have been better".

3.4.2 Key contribution: Model-Predictive Value Expansion (MPVE)

A major innovation of the AC-MPC paper is the improvement of the critic (the network that evaluates the situation). Instead of learning only from observed instantaneous rewards, AC-MPC uses the future predictions of the MPC module. Since the MPC predicts a trajectory of likely future states, the algorithm uses these *imagined* states to refine the estimate of the value $V(s)$. This reduces the variance of learning and accelerates convergence, a concept closely related to world-model-based methods.

3.4.3 Algorithme AC-MPC

Algorithm 1 Actor-Critic Model Predictive Control

1: **Entrée :** Carte de coût neuronale Q_θ, fonction de valeur V_ω
2: **for** $i = 0, 1, 2, \ldots$ **do**
3: Collecte de trajectoires τ :
4: $a_k^\star = \text{diffMPC}(x_k, Q_\theta(x_k), p_k)$
5: $a_k \sim \mathcal{N}(a_k^\star, \Sigma)$
6: Calcul du retour : $R_k = \sum_{t=k}^{T} \gamma^{t-k} r_t$
7: Avantage : $A_k = R_k - V_\omega(x_k)$
8: Mise à jour acteur : $\theta \leftarrow \theta + \alpha A_k \nabla_\theta \log \pi_\theta(a_k|x_k)$ (via diffMPC)
9: Mise à jour critique : $\omega \leftarrow \arg\min_\omega (V_\omega(x_k) - R_k)^2$
10: **end for**
11: **Sortie :** Carte de coût apprise Q_θ

1. Initialization

- **Input:** Initialize the Neural Cost Map (Actor network π_θ) and the Value Function (Critic network V_ω) with random weights.

2. Data Collection Loop

- **Line:** 1
  - **Action:** The main training loop starts.
- **Line:** 2-3
  - **What it does:** The robot interacts with the environment to gather data.
  - **How:** At each time step k:
    1. The **Actor Network** observes state x_k and outputs cost parameters

$$Q(s_k) = \text{diag}(Q(s_k)_{x_1}, \ldots, R(s_k)_{u_1}, \ldots)$$

$$p(s_k) = [p(s_k)_{x_1}, \ldots, p(s_k)_{u_1}, \ldots] \quad \forall k \in 0, \ldots, T$$

    2. The Differentiable MPC solves the optimization problem to find the optimal control u^*_{MPC}, which is minimizing the cost function:

$$u^*_{MPC} = \arg\min_u \sum_{t=0}^{H} (x_t^T Q_k x_t + p_k^T x_t) \quad \text{s.t.} \quad x_{t+1} = f(x_t, u_t)$$

    3. Exploration: We do not execute u^*_{MPC} directly. We sample the actual action u_k from a Gaussian distribution centered on the optimal action with covariance Σ (exploration noise):

$$u_k = u^*_{MPC} + \epsilon, \quad \text{where } \epsilon \sim \mathcal{N}(0, \Sigma)$$

    Note that this is only used during training to explore different actions.
    4. **Storage:** The tuple (x_k, u_k, r_k, x_{k+1}) is stored in the rollout buffer $\mathcal{D}_i$.

3. Value Estimation

- **Line:** 4
  - **What it does:** Calculates the actual return obtained from time step k until the end of the episode.

– Formula:

$$\widehat{R_k} = \sum_{t=k}^{T} \gamma^{t-k} r(x_t, u_t)$$

where $\gamma \in [0, 1[$ is a discount factor that discounts future rewards.

- **Line:** 5
  - **What it does:** Evaluates how much better the action taken was compared to the average expectation.
  - Formula:

$$\widehat{A_k} = \widehat{R_k} - V_\omega(x_k)$$

4. Network Updates

- **Line:** 6
  - **What it does:** Updates the Actor network weights θ to maximize the expected reward.
  - How (The Gradient Update): Since the MPC is differentiable, the gradient flows through the solver, which makes the update possible thanks to the chain rule (differentiation method for a composed function).
  - Formula:

$$\nabla_\theta J(\pi_\theta) = \frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{T} \nabla_\theta \log \pi_\theta(a_k^i \mid s_k^i) A_\omega(s_k^i, a_k^i)$$

- **Line:** 7
  - **What it does:** Updates the Critic network weights ω to better predict future rewards.
  - Formula:

$$\mathcal{L}_V(\omega) = \frac{1}{N} \sum_k |V_\omega(x_k) - \widehat{R_k}|^2$$

$$\omega \leftarrow \omega - \eta \nabla_\omega \mathcal{L}_V$$

Extensions and Considerations

This algorithm can be extended to decrease learning time. However, this method introduces a mismatch between the training distribution and the prediction distribution.

This problem can however be lessened by using the TD k-trick, which consists of training the value function on k-step returns, which helps ensure the training and prediction distributions are closely aligned. This is done by extending the value loss function to:

$$\mathcal{L}_{MPVE} = \frac{1}{H} \sum_{t=0}^{H-1} \left(V_\omega(\widehat{s_t}) - \left(\sum_{k=t}^{H-1} \gamma^{k-t} \widehat{r_k} + \gamma^H \widehat{V_{target}}(\widehat{s_H}) \right) \right)^2$$

With $\widehat{V_H}(s) = \sum_{t=0}^{H-1} \gamma^t \widehat{r_t} + \gamma^H \widehat{V}(s_H)$ and H the horizon.

However, this second method only works when the model is accurate enough up to depth H, otherwise we get the same mismatch problem with other algorithms like TD MPC. Therefore, while this is an update to the first algorithm in most cases, it cannot be used in complex environments where it is difficult to have a clear understanding of physics to create an accurate model.

4 Comparison and Discussion

The analysis of these four methods reveals a convergence in objectives but a divergence in the mechanisms used to ensure alignment between the planner and the policy.

Directionality of Control

The fundamental difference between these methods lies in the directionality of their alignment mechanisms:

- **Policy-constrained Planner (LOOP):** The policy acts as an anchor. The planner is free to optimize but must respect a "Trust Region" defined by the policy. This assumes the policy is a safe baseline and prevents the planner from exploiting model errors (hallucinations).
- **Planner-constrained Policy (TD-M(PC)^2):** The planner is viewed as the teacher. The policy is constrained to stay close to the planner's distribution. This prevents the value function from learning on unrealistic data but may limit the policy's ability to eventually surpass the planner.
- **Bootstrapped Imitation (BMPC):** A cyclical approach. The planner teaches the policy (via imitation), and the policy initializes the planner. There is no hard constraint during the planning phase, but rather a soft guidance via the prior. This allows for a more dynamic evolution where both components improve iteratively.
- **Fused Architecture (AC-MPC):** The distinction fades. The MPC is not an external supervisor or a student; it is a layer within the policy itself. The alignment is implicit because the gradients flow through the planner.

The Core Tension: Consistency vs. Optimality

We see a clear choice between stability and performance. Methods like LOOP play it safe. They force the planner to stay close to what the policy already knows. This ensures that the value estimates remain correct. However, this prevents the system from finding new and better strategies that are very different from the current ones. On the other hand, BMPC is riskier. It trusts the planner completely as an expert. If the model is good, this leads to faster learning. But if the model has errors, the targets are bad, and the policy learns to make mistakes.

Here is the simplified version of the "Computational Implications" section:

Computational Implications

Finally, the computing power needed varies a lot. AC-MPC is slow and can be unstable because it uses complex calculations to learn directly from the solver. BMPC and TD-M(PC)^2 use a different approach: they test thousands of random actions at the same time. This runs fast on graphics cards but requires a huge number of attempts to work well. BMPC uses a smart shortcut called "Lazy Reanalyze" that saves time by reusing old plans instead of starting from scratch.

5 Conclusion

The combination of Model Predictive Control and Reinforcement Learning offers a robust path toward general-purpose robot control. While early attempts treated them as separate modules (Planning for action, RL for value), recent research highlights that their explicit alignment is crucial for success in high-dimensional tasks.

Our analysis shows that there is no single "correct" way to align them. The choice depends on the reliability of the model and the computational budget.

- If the model is uncertain, constraining the planner (LOOP) is safer.
- If the model is good and compute is available, bootstrapping (BMPC) appears to yield the best asymptotic performance.
- If the task requires strictly respecting constraints and dynamics, differentiable layers (AC-MPC) offer the most elegant theoretical solution, though implementation remains complex.

Future work in this field will likely focus on unifying these constraints: can we have a system that dynamically switches between trusting the policy and trusting the planner, based on an online estimate of model uncertainty?

References

[1] Wang, Y., Guo, H., Wang, S., Qian, L., & Lan, X. (2025). Bootstrapped model predictive control. *ICLR 2025*. arXiv preprint arXiv:2503.18871.

[2] Sikchi, H., Zhou, W., & Held, D. (2022). Learning off-policy with online planning. *Conference on Robot Learning (CoRL)*. PMLR.

[3] Lin, H., Wang, P., Schneider, J., & Shi, G. (2025). TD-M(PC)^2: Improving Temporal Difference MPC Through Policy Constraint. arXiv preprint arXiv:2502.03550.

[4] Romero, A., Aljalbout, E., Song, Y., & Scaramuzza, D. (2025). Actor-Critic Model Predictive Control: Differentiable Optimization meets Reinforcement Learning. arXiv:2306.09852.

[5] Hansen, N., Su, H., & Wang, X. (2023). TD-MPC2: Scalable, Robust World Models for Continuous Control. *ICLR*.